Introduction to Proximity Measures

- Definition: Proximity is a measure of how "alike" or "different" two data objects are.
- The Two Pillars:
- Similarity (s): Numerical measure of how alike two objects are. Higher values indicate more similarity (usually scales from 0 to 1)
- Dissimilarity (d): Numerical measure of how different two objects are (e.g., Distance). Lower values indicate more similarity.
- Applications: Clustering, Classification (k-NN), and Recommendation Systems.

Distance Measures (Metric Properties)

- For a measure to be considered a "Metric" (like physical distance), it must satisfy four conditions:
- Non-negativity: $d(x, y) \geq 0$
- Identity of Indiscernibles: $d(x, x) = 0$
- Symmetry: $d(x, y) = d(y, x)$
- Triangle Inequality: $d(x, z) \leq d(x, y) + d(y, z)$

Common Distance Metrics

- Euclidean Distance: The "straight-line" distance between two points.

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (y_i - x_i)^2}$$

Common Distance Metrics

- Manhattan Distance (City Block): The sum of absolute differences (useful for grid-based paths).

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Common Distance Metrics

- Minkowski Distance: The generalized formula for both Euclidean and Manhattan.

$$D(x,y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Non-Metric Similarity Functions

- Sometimes, the "Triangle Inequality" is not useful or valid for specific types of data (like text or biological sequences).
- Cosine Similarity: Measures the angle between two vectors, ignoring their magnitude. High value means they point in the same direction.

  Commonly used in Document Similarity.
- Correlation-based Similarity: Measures how two variables move together (Pearson Correlation).
- Edit Distance (Levenshtein): Measures similarity between strings based on the number of operations to transform one into another.

Proximity Between Binary Patterns

- Binary patterns use 0s and 1s to represent the absence or presence of attributes. To compare them, we use a **Contingency Table**:

	y=1	y=0
x=1	a (Both 1)	b (x is 1, y is 0)
x=0	c (x is 0, y is 1)	d (Both 0)

Binary Similarity Coefficients

- Simple Matching Coefficient (SMC): Matches both 1s and 0s. Useful when "absence" is as important as "presence."

$$SMC = \frac{a + d}{a + b + c + d}$$

- Jaccard Coefficient: Ignores "0-0" matches. Used when the presence of a feature is rare (e.g., market basket analysis).

$$J = \frac{a}{a + b + c}$$

Different Classification Algorithms Based on the Distance Measures

K-Nearest Neighbor (K-NN)

- The most direct application of distance measures. K-NN is a **lazy learner**, meaning it doesn't build a model during training; it simply stores the data and does the work during prediction.

- **How it works:** To classify a new point, the algorithm finds the **k** closest points (neighbors) in the training set and assigns the class that is most common among those neighbors.

- **Distance Role:** The choice of distance (Euclidean, Manhattan, or Minkowski) drastically changes the "neighborhood" shape.

- **Weighted K-NN:** Sometimes, closer neighbors are given more "voting power" than those further away, using a weight of 1/d.

Different Classification Algorithms Based on the Distance Measures

Learning Vector Quantization (LVQ)

- LVQ is a **prototype-based** algorithm. Unlike K-NN, which compares a point to the *entire* dataset, LVQ compares it to a small set of "codebook vectors" (prototypes) that represent each class.
- **How it works:** 1. Prototypes are initialized for each class. 2. During training, if a prototype is the "winner" (closest) for a data point, it moves **toward** the point if the classes match, and **away** if they don't.
- **Proximity Metric:** Usually uses Squared Euclidean distance to define the Voronoi cells (regions of influence) for each prototype.

Different Classification Algorithms Based on the Distance Measures

Radial Basis Function (RBF) Networks

- RBF networks are a type of Artificial Neural Network that uses distance as its primary activation mechanism in the hidden layer.

- **How it works:** Each neuron in the hidden layer represents a "center" point. When an input is fed into the network, the neuron calculates the distance between the input and its center.

- **The "Kernel":** The distance is passed through a Radial Basis Function (typically a **Gaussian function**).

- **Proximity Logic:** The output of the neuron is highest when the distance is zero and drops off as the input moves further away. This effectively creates "bubbles" of high activation around specific points in the feature space.

Different Classification Algorithms Based on the Distance Measures

Support Vector Machines (SVM) with Kernels

- While standard SVMs find a linear boundary, they become "distance-based" classifiers when using **RBF Kernels**.
- **The RBF Kernel:** $K(x, y) = \exp(-\gamma ||x - y||^2)$
- **Distance Role:** The kernel measures the similarity between two points based on their Euclidean distance. In a high-dimensional space, the SVM uses these distance-based similarities to find an "optimal hyperplane" that separates classes with the maximum margin.
- **Non-Metric Kernels:** SVMs can also use non-metric proximity measures (like Jaccard similarity for strings or binary patterns) as custom kernels.

Different Classification Algorithms Based on the Distance Measures

Nearest Centroid (Rocchio Classifier)

- This is the simplest distance-based classifier, often used in text classification.
- **How it works:** 1. Calculate the **Centroid** (mean vector) for each class. 2. Assign a new data point to the class whose centroid is the closest.
- **Common Metric:** Often uses **Cosine Similarity** instead of Euclidean distance, especially when dealing with high-dimensional text data (TF-IDF vectors).

Comparison Table

Algorithm	Primary Measure	Best Used For
K-NN	Euclidean / Manhattan	Small, dense datasets; non-linear patterns.
LVQ	Squared Euclidean	Data compression and fast classification.
RBF Networks	Gaussian (Distance-based)	Function approximation and complex boundaries.
SVM (RBF)	Squared Euclidean	High-dimensional data with no clear linear split.
Nearest Centroid	Cosine Similarity	Text and document classification.

K-Nearest Neighbor (KNN) Classifier

- The K-Nearest Neighbor (KNN) classifier is a supervised machine learning algorithm used for classification and regression. It is a lazy learning and instance-based algorithm because it does not build an explicit model during training. Instead, it stores the training data and makes predictions at runtime.

Key Idea

- An object is classified based on the **majority class of its K nearest neighbors** in the feature space.
- **K** → Number of nearest neighbors considered
- **Distance measure** → Euclidean, Manhattan, or others

Working Principle

- Choose the value of **K**
- Calculate the distance between the test instance and all training instances
- Select the **K nearest neighbors**
- Assign the class based on **majority voting**

Common Distance Measures

Euclidean Distance: The "straight-line" distance between two points.

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (y_i - x_i)^2}$$

Manhattan Distance

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

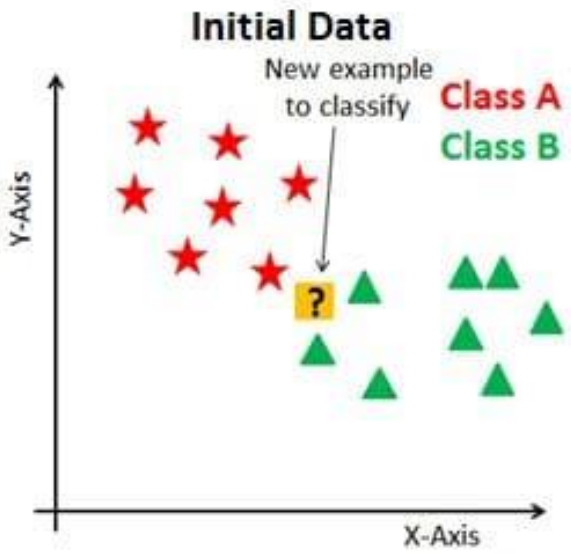
Algorithm: K-Nearest Neighbor Classifier

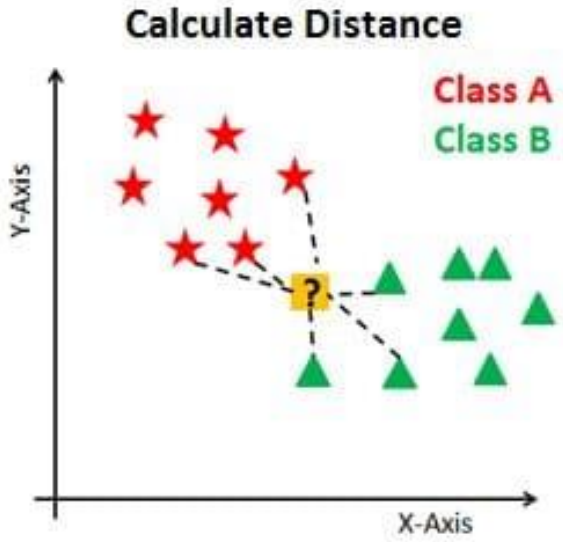
- **Input:**
- Training dataset
- Test sample
- Value of K
- **Output:**
- Predicted class label

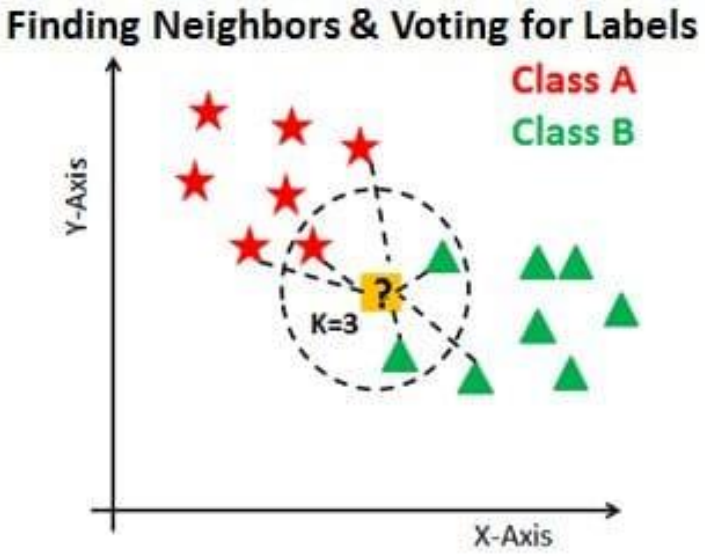
Algorithm: K-Nearest Neighbor Classifier

Steps:

1. Store all training data points.
2. Choose the number of neighbors **K**.
3. Compute the distance between the test point and all training points.
4. Sort distances in ascending order.
5. Select the first **K** nearest neighbors.
6. Count the frequency of class labels among the K neighbors.
7. Assign the class with the **highest frequency** to the test sample.


Initial Data
New example
to classify
Class A
Class B
?
Y-Axis
X-Axis


Calculate Distance
Class A
Class B
?
Y-Axis
X-Axis


Finding Neighbors & Voting for Labels
Class A
Class B
?
K=3
Y-Axis
X-Axis

Example of KNN Classification

Training Data

Data Point	X	Y	Class
A	2	4	Red
B	4	6	Red
C	4	4	Blue
D	6	4	Blue

Test Point
T (5, 5)
K = 3

Step 1: Calculate Euclidean Distance

Point	Distance to T
A	$\sqrt{[(5-2)^2+(5-4)^2]} = 3.16$
B	$\sqrt{[(5-4)^2+(5-6)^2]} = 1.41$
C	$\sqrt{[(5-4)^2+(5-4)^2]} = 1.41$
D	$\sqrt{[(5-6)^2+(5-4)^2]} = 1.41$

Step 2: Select 3 Nearest Neighbors

- B (Red)
- C (Blue)
- D (Blue)

Step 3: Majority Voting

Class	Count
Red	1
Blue	2

Predicted Class: Blue

- **Advantages of KNN**
- Simple and easy to understand
- No training phase
- Works well with multi-class problems
- **Disadvantages of KNN**
- Computationally expensive for large datasets
- Sensitive to noisy data
- Performance depends on choice of K

- **Applications of KNN**
- Pattern recognition
- Medical diagnosis
- Recommendation systems
- Image and text classification

Radius Distance Nearest Neighbor (RNN) Algorithm

- The **Radius Distance Nearest Neighbor (RNN)** algorithm is a **distance-based classification method**.
  Instead of fixing the number of neighbors (as in KNN), RNN considers **all data points within a specified radius (r)** from the query point.

- **Key Idea**
- A test instance is classified using **all training samples that lie within a given distance (radius r)** from it.
- **r (radius)** → Maximum allowable distance
- **Distance measure** → Euclidean, Manhattan, etc.
- **Variable neighbors** → Number of neighbors changes for each query point

Basic Idea

- Choose a **radius (r)**
- For a new data point:
  - Find **all training samples whose distance ≤ r**
  - Assign the class based on:
    - **Majority voting**, or
    - **Weighted voting** (optional)
- If **no points** fall inside the radius, the algorithm may:
  - Increase the radius, or
  - Leave the point unclassified

- **Algorithm: Radius Distance Nearest Neighbor**
- **Input:**
- Training dataset
- Test instance
- Radius value r
- **Output:**
- Predicted class label

Algorithm (Step-by-Step)

- **Input:**
- Training dataset D={(x1,y1),(x2,y2),...,(xn,yn)
- Query point xq
- Radius r
- Distance metric (e.g., Euclidean distance)

Steps:

1. Choose a suitable radius r.
2. For each training point xi, compute distance:

d(xq,xi)

3. Select all points such that:

d(xq,xi)≤r

4. Count class labels of selected points.

5. Assign the class with **maximum frequency**.

6. If no points are found inside the radius, handle it using a predefined rule.

Example

Training Data

Point	Coordinates (x, y)	Class
A	(1, 1)	Red
B	(2, 2)	Red
C	(3, 3)	Blue
D	(6, 6)	Blue

Query Point
Q=(2,3)

Radius
r=2

- **Step 1: Calculate Distances**
- Distance(Q, A):
- sqrt(2−1)2+(3−1)2= sqrt5≈2.23
- Distance(Q, B):
- sqrt(2−2)2+(3−2)2=1

- Distance(Q, C):
- sqrt(3−2)2+(3−3)2=1
- Distance(Q, D):
- sqrt(6−2)2+(6−3)2=5

Step 2: Points Inside Radius (r = 2)

- B → Red
- C → Blue
- (Point A and D are outside the radius)

Step 3: Class Decision

Class	Count
Red	1
Blue	1

Tie case

Possible solutions:

- Reduce radius
- Increase radius
- Use distance-weighted voting
- If distance-weighted voting is used, closer point wins.

Advantages

- No need to predefine number of neighbors
- Adapts well to varying data density
- Simple and intuitive

Disadvantages

- Choosing a good radius is difficult
- Sensitive to noise
- Computationally expensive for large datasets

Comparison: K-NN vs Radius NN

Feature	K-NN	Radius NN
Parameter	k	radius r
Neighbors	Fixed number	Variable
Density handling	Weak	Better
Empty neighbors	Never	Possible